

ozGUI / ozLib

A Python Toolkit for Solving the Ornstein–Zernike Equation

Theory, Formulas, and Usage Reference

August 17, 2026

Abstract

This document is the reference manual for `ozGUI` and `ozLib`, a Python re-implementation of SASfit’s Ornstein–Zernike (OZ) equation solver (`src/sasfit_oz/` in the SASfit source tree). It documents, with exact mathematical formulas, every interaction potential, every closure relation, and every numerical solver algorithm currently implemented, together with the underlying Hankel-transform-based discretisation scheme and the two ways of using the toolkit: as an interactive Tkinter GUI (`ozGUI.py`) and as a plain importable Python library (`ozLib.py`). Formulas throughout were ported and verified directly against SASfit’s own C source (`src/sasfit_oz/sasfit_oz_solver.c`, `src/plugins/*`), and, where an independent literature source was available, cross-checked against it as well; deviations, known limitations, and one deliberately-excluded closure are documented explicitly rather than silently omitted.

Contents

1	Overview	4
2	The Ornstein–Zernike Equation and its Discretisation	4
2.1	The OZ equation	4
2.2	Discrete grid: real space	5
2.3	Discrete Hankel transform	5
2.4	Grand-canonical thermodynamic quantities	5
3	Interaction Potentials	5
3.1	Hard Sphere	6
3.2	Lennard-Jones	6
3.3	Yukawa (screened Coulomb), with optional hard core	6
3.4	Star polymer potential (Likos)	6
3.5	Depletion potential (Oversteegen & Lekkerkerker)	6
3.6	DLVO (electrostatic double-layer + van der Waals)	6
3.7	DLVO + hydration repulsion	7
3.8	Fermi distribution model (Likos)	7
3.9	Generalized Gaussian core model (GGCM- <i>n</i>)	7
3.10	Hard sphere + 3 additive Yukawa tails	7
3.11	Ionic microgel (Likos et al.)	7
3.12	Piecewise-constant hard-sphere shoulder/well (3 segments)	7
3.13	Penetrable sphere model (Likos, Watzlawek, Löwen)	7
3.14	Soft sphere (inverse power law)	8
3.15	Parabolic (Hertzian) sphere	8

3.16	Square well	8
3.17	Sticky hard sphere (Baxter’s adhesive limit)	8
4	Closure Relations	9
4.1	Percus–Yevick (PY)	9
4.2	Hypernetted Chain (HNC)	9
4.3	Reference HNC (RHNC)	9
4.4	Modified HNC (MHNC)	9
4.5	Mean Spherical Approximation (MSA)	9
4.6	Rescaled MSA (RMSA)	9
4.7	Modified MSA (mMSA)	10
4.8	“Symmetric” MSA (SMSA)	10
4.9	Rogers–Young (RY)	10
4.10	HMSA (Zerah–Hansen)	10
4.11	Verlet	10
4.12	Martynov–Sarkisov (MS)	10
4.13	Vompe–Martynov (VM)	11
4.14	BPGG (Ballone–Pastore–Galli–Gazzillo)	11
4.15	CJVM (Charpentier–Jakse extension of VM)	11
4.16	BB (Bomont–Bretonnet)	11
4.17	Kovalenko–Hirata (KH)	11
4.18	Duh–Haymet (DH)	11
4.19	Choudhury–Ghosh (CG)	12
4.20	Euler–Rahman (EuRah) — not offered	12
5	Analytical and Semi-Analytical Cross-Validation Tools	13
5.1	Percus–Yevick hard sphere (Wertheim–Thiele)	13
5.2	Baxter sticky hard sphere (idealised and finite-well)	13
5.3	Sharma–Sharma square well	13
5.4	Rescaled MSA (Hayter–Penfold)	13
5.5	One-Yukawa MSA (Blum–Høye)	14
5.6	ORPA (Pini–Parola–Reatto) for square wells	14
5.7	RFA (Rational Function Approximation)	14
6	Numerical Solver Algorithms	15
6.1	Picard iteration	15
6.2	Anderson acceleration (hand-written)	15
6.3	scipy Anderson / scipy Newton–Krylov	15
6.4	Biggs–Andrews	15
6.5	SUNDIALS KINSOL: Newton–Krylov (GMRES/FGMRES/TFQMR)	15
6.6	SUNDIALS KINSOL: Fixed-Point (Anderson), KIN_FP	16
6.7	Empirical solver comparison	16
7	The ozLib Python Library	17
7.1	Basic usage	17
7.2	Function signature	17
7.3	Example: potential with arguments and a parametrised closure	17
7.4	Example: long-range potential, extended grid	17
8	The ozGUI Application	18
9	Worked Examples: Numerical vs. Analytical Validation	18
9.1	Hard-sphere Percus–Yevick: full validation across φ	18

9.2	Closure comparison at high density	18
9.3	Charged systems: RMSA and the MSA contact-value problem	19
9.4	Behaviour near difficult/critical parameter regimes	21
9.5	Speed	21

1 Overview

ozGUI/ozLib solves the Ornstein–Zernike (OZ) integral equation for a single-component, isotropic fluid of spherically symmetric particles, given

- a pair interaction potential $U(r)$ (Section 3),
- a closure relation connecting the direct correlation function $c(r)$ to the total correlation function $h(r) = g(r) - 1$ (Section 4),
- a volume fraction φ , and
- a numerical solver algorithm that finds the self-consistent fixed point of the resulting nonlinear system (Section 6).

The output is the pair correlation function $g(r)$, the direct correlation function $c(r)$, the static structure factor $S(q)$, and several auxiliary quantities (the cavity function $y(r)$, the bridge function $B(r)$, the indirect correlation function $\Gamma(r) = h(r) - c(r)$, and the reduced potential $U(r)/k_B T$).

Two front ends share the exact same computational core (`ozLib.py`), so results obtained through either are identical by construction:

- `ozGUI.py` — an interactive Tkinter application with potential/closure/solver dropdowns, a 9-tab plot panel, run history, and Save/Load/ASCII/CSV/Excel/PNG export (Section 8).
- `ozLib.py` — a plain Python function, `ozLib.solve(...)`, usable directly in scripts without any GUI dependency (Section 7).

2 The Ornstein–Zernike Equation and its Discretisation

2.1 The OZ equation

For an isotropic, single-component fluid the Ornstein–Zernike equation reads, in real space,

$$h(r) = c(r) + \rho \int c(|\mathbf{r} - \mathbf{r}'|) h(r') d\mathbf{r}', \quad (1)$$

where ρ is the number density and the convolution is over all space. Introducing the indirect correlation function $\Gamma(r) \equiv h(r) - c(r)$, and Fourier–Bessel (Hankel) transforming (exploiting isotropy to reduce the 3D Fourier transform to a 1D sine transform), the OZ equation becomes purely algebraic in reciprocal space,

$$\hat{\Gamma}(q) = \frac{\hat{c}(q)}{1 - \rho \hat{c}(q)} - \hat{c}(q), \quad (2)$$

where $\hat{f}(q)$ denotes the (3D, spherically-symmetric) Fourier transform of $f(r)$. The OZ equation alone relates two unknowns, $c(r)$ and $h(r)$ (or equivalently $\Gamma(r)$); a second, independent relation — the *closure* — is required to close the system. Every closure implemented here has the general form

$$c(r) = f(\Gamma(r), U(r)) \quad (3)$$

for some function f specific to the closure (Section 4). The full problem is therefore a fixed-point problem in $\Gamma(r)$ (or, in an equivalent formulation used by some solvers here, in $c(r)$ and $h(r)$ jointly): starting from a trial $\Gamma(r)$, the closure gives $c(r)$, the OZ equation (Eq. 2) gives a new $\Gamma(r)$, and a solver algorithm (Section 6) drives this map to self-consistency.

2.2 Discrete grid: real space

The real-space grid is a plain equal-spaced grid of $N = \text{numberOfRadialSamplingPoints}$ points,

$$r_i = i \Delta r, \quad i = 1, 2, \dots, N, \quad (4)$$

starting at Δr rather than 0 (division by r appears in the Hankel transform below, so $r = 0$ is avoided by construction rather than special-cased). The hard-core diameter σ is placed at grid index `hardSphereRadiusInPoints` (called n_σ below), i.e.

$$\sigma = n_\sigma \Delta r, \quad \text{total range } r_{\max} = (N - 1) \Delta r = \frac{N - 1}{n_\sigma} \sigma. \quad (5)$$

Both N and n_σ are configurable (constructor keyword arguments `numberOfRadialSamplingPoints`, `hardSphereRadiusInPoints` on every solver class, and identically-named arguments on `ozLib.solve()`); the historical defaults are $N = 4096$, $n_\sigma = 100$, giving $r_{\max} \approx 41 \sigma$ and $\Delta r = 0.01 \sigma$. Increasing N at fixed n_σ extends the range without losing near-contact resolution — important for slowly-decaying, long-range potentials (e.g. small- κ Yukawa/DLVO tails), where a too-short grid truncates the potential before it has actually decayed to zero, distorting $S(q)$ at low q (this was verified directly: a $4\times$ extension of the default grid reduced a Yukawa potential's residual value at the grid edge from $\mathcal{O}(10^{-4}) k_B T$ to $\mathcal{O}(10^{-10}) k_B T$, and changed $S(q \rightarrow 0)$ by about 33%).

2.3 Discrete Hankel transform

For an isotropic function $f(r)$ the 3D Fourier transform reduces to

$$\hat{f}(q) = \frac{4\pi}{q} \int_0^\infty r f(r) \sin(qr) dr, \quad (6)$$

which is implemented here as a type-I discrete sine transform (DST) of $r_i f(r_i)$, following the same construction documented in SASfit's own theory notes (equations 5.18/5.19 there):

$$\hat{f}(q_k) = \frac{2\pi \Delta r^2}{\Delta q} \cdot \frac{\text{DST-I}[r_i f(r_i)]_k}{k}, \quad \Delta q = \frac{\pi}{(N + 1) \Delta r}, \quad q_k = k \Delta q. \quad (7)$$

The inverse transform is the same operation up to a normalisation factor ($\Delta q^3 / [(2\pi)^3 \Delta r^3]$), since the Hankel transform pair is (up to constants) self-inverse. Both directions are implemented once, in `0ZfixpointOperator.hankelTransform()/inverseHankelTransform()`, and shared by every closure and every solver.

2.4 Grand-canonical thermodynamic quantities

For the hard-sphere/Percus–Yevick case, the exact grand-canonical compressibility-route structure factor at $q = 0$ is available in closed form (used as an internal consistency check, e.g. in RMSA's own bisection, see Section 4.6):

$$S(q=0) = \frac{(1 - \varphi)^4}{(1 + 2\varphi)^2}. \quad (8)$$

3 Interaction Potentials

Every potential is specified through its Boltzmann factor $\exp[-U(r)/k_B T]$ (stored as `boltzmannOfP2Potential` evaluated on the grid described above; all energies are in $k_B T$ units and all lengths in units where the hard-core diameter is $\sigma = n_\sigma \Delta r$). Potentials that support the repulsive/attractive potential split needed by switching-function closures (Rogers–Young, HMSA, and the Gstar-based bridge closures of Section 4) additionally populate `repulsivePartOfP2Potential/attractivePartOfP2Potential`. All 17 potentials below are available via `setPotentialByName(name, *args)` or directly as `set<Name>Potential(...)`.

3.1 Hard Sphere

$$\exp[-U(r)/k_B T] = \begin{cases} 0 & r < \sigma \\ 1 & r \geq \sigma. \end{cases} \quad (9)$$

3.2 Lennard-Jones

$$U(r)/k_B T = 4\varepsilon \left[\left(\frac{\sigma}{r} \right)^{12} - \left(\frac{\sigma}{r} \right)^6 \right], \quad (10)$$

with ε in $k_B T$ units. The potential is split at its minimum ($r_{\min} = 2^{1/6}\sigma$) into a repulsive part ($r < r_{\min}$) and an attractive part ($r \geq r_{\min}$), for use by switching-function closures.

3.3 Yukawa (screened Coulomb), with optional hard core

$$U(r)/k_B T = K \frac{\sigma}{r} e^{-r/\lambda}, \quad (11)$$

where λ is the screening length (in units of σ) and K the contact interaction strength; a hard core can optionally be added (`doAddHardSphere`), in which case the exponential and $1/r$ factors are shifted independently as in SASfit's own construction, and the potential is set to ∞ (Boltzmann factor 0) for $r < \sigma$.

3.4 Star polymer potential (Likos)

$$U(r)/k_B T = \begin{cases} \frac{5}{18} f^{3/2} \left[-\ln \frac{r}{\sigma} + \frac{1}{1 + \sqrt{f}/2} \right] & r \leq \sigma \\ \frac{5}{18} f^{3/2} \frac{1}{1 + \sqrt{f}/2} \frac{\sigma}{r} \exp \left[-\frac{\sqrt{f}(r - \sigma)}{2\sigma} \right] & r > \sigma, \end{cases} \quad (12)$$

where $f = \text{NumberOfArms}$ is the number of arms of the star polymer.

3.5 Depletion potential (Oversteegen & Lekkerkerker)

Two-sphere overlap-volume depletion potential between big spheres (diameter $\sigma_1 = \sigma$, the species this OZ solve is for) induced by small depletant spheres (diameter $\sigma_2 = \text{sigmaRatio} \cdot \sigma_1$, number density ρ_2 set via the depletant volume fraction `phi2`):

$$U(r)/k_B T = -\rho_2 \frac{\pi \sigma_1^3 (1 + r_t)^3}{6} \left[1 - \frac{3r}{2(1 + r_t)\sigma_1} + \frac{r^3}{2(1 + r_t)^3 \sigma_1^3} \right], \quad \sigma_1 \leq r < \sigma_1 + \sigma_2, \quad (13)$$

with $r_t = \sigma_2/\sigma_1$, and $U = 0$ for $r \geq \sigma_1 + \sigma_2$; hard core for $r < \sigma_1$.

3.6 DLVO (electrostatic double-layer + van der Waals)

$$U(r)/k_B T = U_{\text{el}}(r) + U_{\text{vdW}}(r), \quad r > \sigma, \quad (14)$$

$$U_{\text{el}}(r) = \frac{L_B Z^2}{(1 + \kappa a)^2} \frac{e^{-\kappa(r - \sigma)}}{r}, \quad U_{\text{vdW}}(r) = -\frac{A}{6} \left[\frac{2a^2}{r^2 - 4a^2} + 2 \left(\frac{a}{r} \right)^2 + \ln \left(1 - 4 \left(\frac{a}{r} \right)^2 \right) \right], \quad (15)$$

where $a = \sigma/2$, κ the inverse Debye screening length, Z the macro-ion charge, L_B the Bjerrum length, and A the effective Hamaker constant. U_{vdW} 's genuine divergence at $r = \sigma$ is excluded from the computation itself (not merely masked afterward), since the grid places a real point at exactly $r = \sigma$.

3.7 DLVO + hydration repulsion

As Section 3.6, plus a short-range exponential hydration term,

$$U(r)/k_B T = U_{\text{DLVO}}(r) - 2G_{HY} e^{-(r-\sigma)/D_H}, \quad r > \sigma, \quad (16)$$

with hydration strength G_{HY} and decay length D_H .

3.8 Fermi distribution model (Likos)

$$U(r)/k_B T = \varepsilon \frac{1 + e^{-\sigma/\xi}}{1 + e^{(r-\sigma)/\xi}}. \quad (17)$$

The limit $\xi \rightarrow 0$ reduces to the penetrable sphere model (below), and is handled as a special case when $\xi = 0$ is passed exactly.

3.9 Generalized Gaussian core model (GGCM- n)

$$U(r)/k_B T = \varepsilon \exp\left[-\alpha \left(\frac{r}{\sigma}\right)^n\right]. \quad (18)$$

3.10 Hard sphere + 3 additive Yukawa tails

$$U(r)/k_B T = -\frac{\sigma}{r} \sum_{i=1}^3 K_i \exp\left[-\frac{r-\sigma}{\sigma \lambda_i}\right], \quad r \geq \sigma, \quad (19)$$

with hard core for $r < \sigma$; each (K_i, λ_i) pair may be set to zero to recover fewer effective Yukawa terms.

3.11 Ionic microgel (Likos et al.)

A multi-term analytical potential for a charged microgel sphere of charge Z , dielectric constant ED , and inverse screening length κ_{Π}/σ inside the gel, following Likos et al. (2011); the full piecewise expression (different polynomial/hyperbolic forms inside vs. outside the gel radius) is implemented exactly as in `sasfit_oz_potential_ionic_microgel.c` and is reproduced verbatim in the source rather than restated here, since it spans several auxiliary terms without a single compact closed form.

3.12 Piecewise-constant hard-sphere shoulder/well (3 segments)

$$U(r)/k_B T = \begin{cases} \varepsilon_1 & \sigma \leq r < \sigma + \delta_1 \\ \varepsilon_2 & \sigma + \delta_1 \leq r < \sigma + \delta_1 + \delta_2 \\ \varepsilon_3 & \sigma + \delta_1 + \delta_2 \leq r < \sigma + \delta_1 + \delta_2 + \delta_3 \\ 0 & \text{otherwise (outside all three steps),} \end{cases} \quad (20)$$

hard core for $r < \sigma$; each ε_j may be positive (shoulder) or negative (well). This is the general $n = 3$ piecewise-constant family used as an independent cross-check for the RFA analytical solver (Section 5.7).

3.13 Penetrable sphere model (Likos, Watzlawek, Löwen)

$$U(r)/k_B T = \begin{cases} \varepsilon & r \leq \sigma \\ 0 & r > \sigma. \end{cases} \quad (21)$$

3.14 Soft sphere (inverse power law)

$$U(r)/k_B T = \varepsilon \left(\frac{\sigma}{r} \right)^n. \quad (22)$$

3.15 Parabolic (Hertzian) sphere

$$U(r)/k_B T = \begin{cases} \varepsilon \left(1 - \frac{r}{\sigma} \right)^{5/2} & r \leq \sigma \\ 0 & r > \sigma. \end{cases} \quad (23)$$

3.16 Square well

$$U(r)/k_B T = \begin{cases} \infty & r < \sigma \\ \varepsilon & \sigma \leq r \leq \sigma + \delta \\ 0 & r > \sigma + \delta, \end{cases} \quad (24)$$

with well width δ (well outer range $\lambda = 1 + \delta/\sigma$ in the notation of Section 5.3); $\varepsilon < 0$ for an attractive well, $\varepsilon > 0$ for a repulsive shoulder.

3.17 Sticky hard sphere (Baxter's adhesive limit)

$$U(r)/k_B T = \begin{cases} \infty & r < \sigma \\ \ln \left(\frac{12 \tau \delta}{\sigma + \delta} \right) & \sigma \leq r \leq \sigma + \delta \\ 0 & r > \sigma + \delta, \end{cases} \quad (25)$$

using Baxter's stickiness parameter τ (small τ = sticky, large τ = hard-sphere limit) and a finite well width δ — the same τ convention as SASfit's own `robertus.shs` plugin, giving a natural point of comparison between this (monodisperse, OZ-based) solve and that (polydisperse, PY-only, multicomponent) one.

4 Closure Relations

Every closure below expresses $c(r)$ as a function of $\Gamma(r)$ (and, for the switching-function and Gstar-based closures, the potential's own repulsive/attractive split) and is dispatched through `OZfixpointOperator.update_c()`. Formulas were cross-checked directly against SASfit's own `sasfit_oz_solver.c` case statements (not re-derived from a textbook), using that file's own notation: $\text{EN}(r) = \exp[-U(r)/k_B T]$, $G(r) = \Gamma(r)$. Where a closure needs an extra scalar parameter (shared storage slot `self.alpha`, matching the C code's own `sBPGG/aCJVM/fBB` `#defines` onto the same underlying variable), this is noted explicitly. **19 closures are offered**; a 20th (Euler–Rahman) exists in the source but is deliberately excluded from both the GUI and `ozLib.solve()`'s validated closure list — see Section 4.20 for why.

4.1 Percus–Yevick (PY)

$$c(r) = \text{EN}(r) [1 + \Gamma(r)] - \Gamma(r) - 1. \quad (26)$$

Equivalent to the standard PY form $c(r) = (e^{-U/k_B T} - 1)y(r)$ with cavity function $y(r) = g(r)e^{U/k_B T}$.

4.2 Hypernetted Chain (HNC)

$$c(r) = \text{EN}(r) e^{\Gamma(r)} - \Gamma(r) - 1. \quad (27)$$

4.3 Reference HNC (RHNC)

Needs a reference-system sub-solve: a bare hard sphere of the same σ is first solved with the Martynov–Sarkisov closure (Section 4.12), giving reference cavity/indirect-correlation functions $g_0(r)$, $\Gamma_0(r)$. Writing the actual potential's perturbation as $\Delta U(r)/k_B T = -\ln \text{EN}(r)$ (zero inside the reference hard core), the closure is

$$c(r) = g_0(r) \exp[(\Gamma(r) - \Gamma_0(r)) - \Delta U(r)/k_B T] - \Gamma(r) - 1. \quad (28)$$

This reduces to the same hard-core exclusion every other hard-sphere closure uses automatically, since $g_0(r) \equiv 0$ inside the reference hard core. Orchestrated by `OZsolver.solveRHNC()`.

4.4 Modified HNC (MHNC)

HNC plus a fixed, purely analytical Percus–Yevick hard-sphere bridge function $B_{\text{PY}}(r; \eta)$ (no reference-system sub-solve needed, unlike RHNC, since B_{PY} has a closed form):

$$c(r) = \text{EN}(r) \exp[\Gamma(r) + B_{\text{PY}}(r; \eta)] - \Gamma(r) - 1, \quad (29)$$

with the packing-fraction-like parameter η supplied by the user (`doMHNCclosure(eta)`).

4.5 Mean Spherical Approximation (MSA)

$$c(r) = \begin{cases} -[\Gamma(r) + 1] & r < \sigma \quad (\text{forces } g(r)=0) \\ -U(r)/k_B T = \ln \text{EN}(r) & r \geq \sigma. \end{cases} \quad (30)$$

4.6 Rescaled MSA (RMSA)

Identical formula to MSA, but the inside/outside switch is driven by an *apparent* hard-core gate $g_{4g}(r) \in \{0, 1\}$ rather than the bare potential's own hard core:

$$c(r) = \begin{cases} -[\Gamma(r) + 1] & g_{4g}(r) = 0 \\ \ln \text{EN}(r) & g_{4g}(r) = 1. \end{cases} \quad (31)$$

`OZsolver.solveRMSA()` first solves plain MSA; if the resulting $g(\sigma^+) < 0$ (an unphysical result plain MSA can give for strongly-coupled systems), it re-solves with the apparent hard core bisected outward, grid point by grid point, until g at the new apparent contact point is non-negative. This is the same Hayter–Penfold-style rescaling idea underlying Section 5.4’s semi-analytical RMSA solver, applied here inside the general numerical OZ solve instead of via the closed-form Hayter–Penfold machinery.

4.7 Modified MSA (mMSA)

As MSA, but using the Mayer function outside the core instead of the bare potential:

$$c(r) = \begin{cases} -[\Gamma(r) + 1] & r < \sigma \\ \text{EN}(r) - 1 & r \geq \sigma. \end{cases} \quad (32)$$

4.8 “Symmetric” MSA (SMSA)

Using $\Gamma^*(r) = \Gamma(r) - U_{\text{attr}}(r)/k_B T$ (the attractive-part-subtracted indirect correlation function, shared with several closures below):

$$c(r) = \exp[U_{\text{attr}}(r)/k_B T] [\Gamma^*(r) + 1] - \Gamma(r) - 1. \quad (33)$$

4.9 Rogers–Young (RY)

Interpolates between PY ($\alpha \rightarrow \infty$) and HNC ($\alpha \rightarrow 0$) via a switching function $f(r) = 1 - e^{-\alpha r}$:

$$c(r) = \text{EN}(r) \left[1 + \frac{e^{f(r)\Gamma(r)} - 1}{f(r)} \right] - \Gamma(r) - 1, \quad (34)$$

with α supplied by the user (`doRYclosure(alpha)`).

4.10 HMSA (Zerah–Hansen)

Same switching-function structure as RY, but using $\Gamma^*(r)$ in the switching term and the full Boltzmann factor split into repulsive/attractive parts:

$$c(r) = \exp[-U_{\text{rep}}(r)/k_B T] \left[1 + \frac{e^{f(r)\Gamma^*(r)} - 1}{f(r)} \right] - \Gamma(r) - 1, \quad (35)$$

with α (in $f(r) = 1 - e^{-\alpha r}$) supplied by the user (`doHMSAclosure(alpha)`).

4.11 Verlet

Empirical bridge function, no reference system or extra parameter:

$$B(r) = \frac{-\Gamma(r)^2}{2[1 + 0.8\Gamma(r)]}, \quad c(r) = \text{EN}(r) e^{\Gamma(r)+B(r)} - \Gamma(r) - 1. \quad (36)$$

4.12 Martynov–Sarkisov (MS)

$$B(r) = \text{sgn}(1+2\Gamma) \sqrt{|1 + 2\Gamma(r)|} - \Gamma(r) - 1, \quad c(r) = \text{EN}(r) e^{\Gamma(r)+B(r)} - \Gamma(r) - 1. \quad (37)$$

Used standalone, and as the reference-system closure for RHNC (Section 4.3).

4.13 Vompe–Martynov (VM)

As MS, but using $\Gamma^*(r)$ in place of $\Gamma(r)$ inside the square root:

$$B(r) = \text{sgn}(1+2\Gamma^*)\sqrt{|1+2\Gamma^*(r)|} - \Gamma^*(r) - 1, \quad c(r) = \text{EN}(r) e^{\Gamma(r)+B(r)} - \Gamma(r) - 1. \quad (38)$$

4.14 BPGG (Ballone–Pastore–Galli–Gazzillo)

Power-law bridge function with user parameter s (`doBPGGclosure(s)`, $s \neq 0$; SASfit's own `sBPGG`):

$$B(r) = \text{sgn}(1+s\Gamma) |1+s\Gamma(r)|^{1/s} - \Gamma(r) - 1, \quad c(r) = \text{EN}(r) e^{\Gamma(r)+B(r)} - \Gamma(r) - 1. \quad (39)$$

4.15 CJVM (Charpentier–Jakse extension of VM)

User parameter a (`doCJVMclosure(a)`, $a \neq 0$; SASfit's own `aCJVM`):

$$B(r) = \frac{1}{2a} \left[\text{sgn}(1+4a\Gamma^*)\sqrt{|1+4a\Gamma^*(r)|} - 1 - 2a\Gamma^*(r) \right] - \Gamma^*(r) - 1, \quad (40)$$

$$c(r) = \text{EN}(r) e^{\Gamma(r)+B(r)} - \Gamma(r) - 1. \quad (41)$$

4.16 BB (Bomont–Bretonnet)

User parameter f (`doBBclosure(f)`; SASfit's own `fBB`):

$$B(r) = \text{sgn}(1+2\Gamma^*+f(\Gamma^*)^2)\sqrt{|1+2\Gamma^*(r)+f\Gamma^*(r)^2|} - \Gamma^*(r) - 1, \quad (42)$$

$$c(r) = \text{EN}(r) e^{\Gamma(r)+B(r)} - \Gamma(r) - 1. \quad (43)$$

4.17 Kovalenko–Hirata (KH)

$$c(r) = \text{EN}(r) e^{\Gamma(r)} - \Gamma(r) - 1. \quad (44)$$

Remark 1. Reading SASfit's own `case KH`: block directly (rather than assuming the textbook definition) revealed that its $c(r)/g(r)$ formula there is identical to HNC's own. The textbook KH closure is normally a piecewise-linearised bridge function ($g(r) = \exp(\Gamma - U/k_B T)$ where this argument is ≤ 0 , else $g(r) = 1 + \Gamma - U/k_B T$, designed to avoid HNC's own divergence risk at high density); SASfit's C code computes this piecewise bridge function too, but only as a separate diagnostic output that is never fed back into $c(r)$ in that same closure branch. This is reproduced here exactly as the C code actually computes it, quirk included, since the purpose of this port is to reproduce SASfit's own numbers, not to independently re-derive the textbook closure. Verified numerically: KH and HNC give identical $g(r)$ to machine precision for every potential/density tested.

4.18 Duh–Haymet (DH)

A rational-function bridge approximation:

$$B(r) = \frac{-\Gamma^*(r)^2}{2 \left[1 + \frac{5\Gamma^*(r)+11}{7\Gamma^*(r)+9} \Gamma^*(r) \right]}, \quad c(r) = \text{EN}(r) e^{\Gamma(r)+B(r)} - \Gamma(r) - 1. \quad (45)$$

Independently verified against Pihlajamaa & Janssen (2024) [2], Table 1: $b(r) = -\frac{1}{2}\gamma^2(r)[1 + \gamma(r)(5\gamma(r)+11)/(7\gamma(r)+9)]^{-1}$, an exact match. *Known gap*: SASfit's own `case DH`: additionally special-cases the Γ^* formula specifically when the active potential is Lennard-Jones (a narrower numerical-stability tweak for that potential's own r^{-12} term, not a different closure formula); this special case is not replicated here.

4.19 Choudhury–Ghosh (CG)

A piecewise bridge function with a density-dependent coefficient:

$$B(r) = \begin{cases} \frac{-\frac{1}{2}\Gamma^*(r)^2}{1 + \zeta(\varphi)\Gamma^*(r)} & \Gamma^*(r) > 0 \\ -\frac{1}{2}\Gamma^*(r)^2 & \Gamma^*(r) \leq 0, \end{cases} \quad \zeta(\varphi) = 1.0175 - 0.275 \frac{6\varphi}{\pi}, \quad (46)$$

$$c(r) = \text{EN}(r) e^{\Gamma(r)+B(r)} - \Gamma(r) - 1. \quad (47)$$

Independently verified against Pihlajamaa & Janssen (2024) [2], Table 1: $\zeta(\rho) = 1.0175 - 0.275 \rho \sigma^3$, and since $\rho \sigma^3 = 6\varphi/\pi$ for spheres, this is an exact match (including the same $\Gamma^* \leq 0$ piecewise branch).

4.20 Euler–Rahman (EuRah) — not offered

Based on Eu & Rah, *J. Chem. Phys.* **111**, 3327 (1999) [1], “A closure for the Ornstein–Zernike relation that gives rise to the thermodynamic consistency.” Unlike every closure above, $c(r)$ here is not a local function of $\Gamma(r)$ alone: it is fixed externally to an array $c_{\text{EuRah}}(r)$ that must itself satisfy an outer self-consistency condition,

$$c_{\text{EuRah}}(r) = \frac{1}{36} [\text{MAYER}(r)] \left[36y(r) + 18\varphi \frac{\partial y}{\partial r} + 12r \frac{\partial y}{\partial \varphi} + 6\varphi r \frac{\partial^2 y}{\partial r \partial \varphi} \right], \quad (48)$$

where $\text{MAYER}(r) = \text{EN}(r) - 1$, $y(r)$ is the cavity function $y(r) = g(r)e^{U(r)/k_B T}$ evaluated at the trial c_{EuRah} , and the density derivatives are evaluated by re-solving the OZ equation at $\varphi \pm 0.01\varphi$ for the same trial c_{EuRah} . Structurally, writing $36y(r)$ as the dominant term, Eq. 48 is exactly “PY closure plus systematic density/spatial-derivative correction terms” ($c_{\text{PY}} = \text{MAYER} \cdot y$), which is the expected shape for a closure designed to restore thermodynamic consistency to PY.

This closure is implemented (`OZsolver.solveEuRah()`, `doEuRahClosure()`) but is deliberately excluded from `ozLib.CLOSURE.SETTERS` and therefore from both the GUI dropdown and `ozLib.solve()`’s validated closure list. Extensive testing found a genuine exponential positive-feedback instability intrinsic to Eq. 48’s own structure: inside the hard core, $y(r) = e^{\Gamma(r)}$, and $\Gamma(r)$ itself depends on $c_{\text{EuRah}}(r)$ through the OZ equation, so a more negative trial c_{EuRah} produces an exponentially larger $y(r)$, which (via the dominant $36y(r)$ term) produces an even more negative c_{EuRah} on the next iteration. This defeated every numerical strategy attempted: Newton–Krylov (`scipy.optimize.newton.krylov`), damped fixed-point iteration (multiple damping factors), density continuation with warm restarts, dense-Jacobian Newton methods (`fsolve/hybr`, Levenberg–Marquardt, Broyden1/2, `df-sane`), a trust-region-style adaptive step limiter, and SUNDIALS’ own KINSOL (via `sundials4py`) with this project’s best-validated tuning parameters (Section 6.5) — the last of these even produced low-level crashes (segfault/bus error) rather than merely failing to converge. The formula’s overall structure was independently verified as plausible (see above), and the literature explicitly notes that this class of “second-order” closure is excluded from standard closure-comparison studies for “numerical complexity” [2]; the instability found here is consistent with that being a genuine property of the theory rather than an implementation error.

5 Analytical and Semi-Analytical Cross-Validation Tools

Independent of the general numerical OZ solve, several closed-form or semi-analytical structure-factor solutions are implemented as cross-checks. These are consulted throughout this project to validate the numerical solvers and closures above, rather than offered as alternative closures themselves.

5.1 Percus–Yevick hard sphere (Wertheim–Thiele)

The classic closed-form PY hard-sphere solution (Naegele’s notation),

$$S(q) = \frac{1}{X(q)^2 + Y(q)^2}, \quad X = 1 - 12\varphi(Af_1 + Bf_2), \quad Y = -12\varphi(Af_3 + Bf_4), \quad (49)$$

with $A = (1 + 2\varphi)/(1 - \varphi)^2$, $B = (1 + \varphi/2)/(1 - \varphi)^2$, and $f_1, \dots, f_4$ standard trigonometric combinations of $y = q\sigma$. Verified directly against SASfit’s own `GPY()` formula; found the existing Python implementation to be numerically *more* robust than SASfit’s own C code at very low q (SASfit’s small-argument Taylor threshold is far tighter than where the raw trigonometric formula actually loses precision).

5.2 Baxter sticky hard sphere (idealised and finite-well)

Baxter’s (1968) closed-form adhesive-sphere solution using stickiness parameter τ , ported directly from `sasfit_sq_sticky_hard_sphere.c`, plus a finite-well generalisation (`sasfit_sq_sticky_hard_sphere` for a well of finite width δ — the physically correct comparison for this project’s own finite- δ `StickyHardSphere` potential (Section 3.17), since the idealised $\delta \rightarrow 0$ limit is not the same physical system.

5.3 Sharma–Sharma square well

The classic first-order perturbative PY square-well solution (Sharma & Sharma, *Physica A* **89**, 213 (1977)), ported from `sasfit_sq_square_well.potential.c`:

$$S(q) = \frac{1}{1 - C(q)}, \quad C(q) = \frac{-24\varphi}{\kappa^6} \left[\alpha \kappa^3 (\sin \kappa - \kappa \cos \kappa) + \beta \kappa^2 (2\kappa \sin \kappa - (\kappa^2 - 2) \cos \kappa - 2) \right. \\ \left. + \gamma ((4\kappa^3 - 24\kappa) \sin \kappa - (\kappa^4 - 12\kappa^2 + 24) \cos \kappa + 24) - \varepsilon \kappa^3 (\sin \lambda \kappa - \lambda \kappa \cos \lambda \kappa + \kappa \cos \kappa - \sin \kappa) \right], \quad (51)$$

with $\kappa = q\sigma$, well outer range $\lambda = 1 + \delta/\sigma$, and α, β, γ the same PY-like density coefficients as the hard-sphere solution above. Being a first-order expansion, this closed-form solution degrades for strong/wide wells (verified: max. deviation from the full numerical solve rose from ~ 0.005 at $\varepsilon = -0.02, \delta = 0.02\sigma$ to ~ 0.33 at $\varepsilon = -0.5, \delta = 0.3\sigma$, at $\varphi = 0.15$), which is the expected behaviour of a perturbative expansion approaching the edge of its own validity.

5.4 Rescaled MSA (Hayter–Penfold)

The full Hayter–Penfold RMSA solution for a hard-core Yukawa/screened-Coulomb potential, compiled as a standalone C library (`rmsa_c_source/`, from `src/plugins/RMSA/{rmsa.c,polyroots.c,rmsa_ph` and wrapped via `ctypes` (`rmsaWrapper.py`), reusing SASfit’s own quartic-root selection and small- κ Taylor-series logic rather than re-implementing it. Validated: the weak-coupling limit reproduces the exact PY hard-sphere contact value $g(\sigma^+) = (1 + \varphi/2)/(1 - \varphi)^2$.

5.5 One-Yukawa MSA (Blum–Høye)

A semi-analytical Yukawa-MSA solution, compiled as a standalone C library (`oneyukawa_c_source/`, from `src/plugins/twoyukawa/`) including a full Jenkins–Traub complex polynomial root-finder (`2Y_cpoly.c`) and an FFT-based pair-correlation-function evaluation (`2Y_PairCorrelation.c`) used to select the physically correct root among up to four candidates. Cross-validated against the general numerical OZ solve (HS3Yukawa potential + MSA closure) to within $\sim 2\%$ mean / $\sim 5\%$ max deviation, concentrated at low q and shrinking rapidly at higher q — the same order of residual found for the exact PY hard-sphere case, consistent with a numerical-grid-vs-exact discrepancy rather than an error in either implementation.

5.6 ORPA (Pini–Parola–Reatto) for square wells

A self-consistent optimised random-phase approximation specific to square-well potentials (Pini, Parola & Reatto, *Mol. Phys.* **100**, 1507 (2002)), compiled as a standalone C library (`orpa_c_source/`) using GSL’s derivative-free multivariate root solver with continuation in the well depth. At a strong-well test case ($\varphi = 0.15$, $\varepsilon = -0.5k_B T$, $\delta = 0.3\sigma$) where the Sharma–Sharma perturbative formula (Section 5.3) showed max. deviation 0.33 from the numerical solve, ORPA gave max. deviation 0.057 — roughly $6\times$ closer, consistent with ORPA being a genuinely non-perturbative theory.

5.7 RFA (Rational Function Approximation)

Santos, Yuste & López de Haro’s rational-function approximation for *arbitrary* piecewise-constant potentials (*Condens. Matter Phys.* **15**, 23602 (2012)), compiled as a standalone C library (`rfa_c_source/`) with a numerically-safe small- q series expansion. Because it handles an arbitrary number of piecewise-constant steps, this is the only analytical tool validated against both the single-well `SquareWell` potential ($n = 1$; max. deviation 0.025 on the same strong-well case above, the closest of the three square-well cross-checks) *and* the genuinely multi-step `PiecewiseConstantHS` potential ($n = 3$ shoulder-well-shoulder; max. deviation 0.0056/mean 0.0028), a cross-check no other tool here can perform.

6 Numerical Solver Algorithms

Given a closure (Section 4), the OZ equation (Eq. 2) reduces the physics problem to a fixed-point problem $\Gamma = \mathcal{F}(\Gamma)$, where $\mathcal{F}$ composes the closure and the Hankel-transform-based OZ update. Eight solver algorithms are available, offering a range of speed/reliability tradeoffs (Section 6.7 compares them empirically).

6.1 Picard iteration

Plain, undamped fixed-point iteration, $\Gamma_{n+1} = \mathcal{F}(\Gamma_n)$, until the relative change in $\|\Gamma_n\|$ falls below a convergence criterion. Simple and reliable for easy problems; can fail to converge for harder ones (e.g. dense hard spheres, strongly-charged systems).

6.2 Anderson acceleration (hand-written)

A direct Python implementation of Anderson mixing: the next iterate is a linear combination of the last m residuals, with mixing coefficients chosen by a small least-squares problem, extrapolating the fixed-point iteration rather than taking it verbatim. Confirmed independently reliable in most cases, but found (Section 6.7) to occasionally converge to a spurious answer on very hard problems (strongly-charged MSA) without raising any error — silently disagreeing with cross-validated Newton–Krylov solutions by a wide margin in that regime.

6.3 scipy Anderson / scipy Newton–Krylov

Thin wrappers around `scipy.optimize.anderson/newton_krylov` respectively, applied to $\mathcal{F}$. Reliable across the full range of problems tested here, though can be markedly slower than the alternatives below on both easy and hard problems.

6.4 Biggs–Andrews

A hand-written acceleration scheme, ported directly from `sasfit_oz_solver.c`’s own `CASE BIGGS-ANDREWS`: block (dispatched there through the same `OZ_solver_by_iteration()` as Picard — i.e. this is *not* SUNDIALS-based, despite the KINSOL-adjacent naming of its own tuning parameter). After two unaccelerated warm-up steps, each further step:

1. computes a Barzilai–Borwein-style spectral step size $\alpha = \beta/\gamma$ (or $\text{sgn}(\beta/\gamma)\sqrt{|\beta/\gamma|}$), with $\beta = \langle g_n, g_{n-1} \rangle$, $\gamma = \langle g_{n-1}, g_{n-1} \rangle$ inner products of consecutive fixed-point residuals $g_k = \mathcal{F}(x_k) - x_k$;
2. clamps α to the Nesterov momentum bound $[-\frac{n-1}{n+2}, \frac{n-1}{n+2}]$;
3. extrapolates using order `|KINSetMAA|` (0: none; 1: heavy-ball momentum $y = x + \alpha(x - x_{\text{prev}})$; ≥ 2 : adds a second-order term $\frac{\alpha^2}{2}(x - 2x_{\text{prev}} + x_{\text{prevprev}})$).

Validated: order 0 reproduces plain Picard almost exactly (354 vs. 353 steps on hard-sphere/PY at $\varphi = 0.3$); orders 1/−2 roughly halve the iteration count on that same case; every configuration agrees with the independently validated GMRES solver to $\sim 10^{-12}$.

6.5 SUNDIALS KINSOL: Newton–Krylov (GMRES/FGMRES/TFQMR)

A genuine Newton–Krylov method via SUNDIALS’ own KINSOL, through its official Python bindings (`sundials4py`), using the `KIN_LINESEARCH` strategy with a selectable linear solver. Tuning parameters were matched exactly to SASfit’s own “configure OZ solver” dialog defaults (shared directly by a user of this project): `KINSetMaxNewtonStep= 100N` (not just N),

`KINSetFuncNormTol= 10-10`, `KINSetScaledStepTol= 10-13`, `KINSetMaxRestarts= 10` (GMRES/FGMRES only), `KINSetEtaForm= KIN_ETACHOICE1`. This made a confirmed, material difference: a strongly-charged DLVO+MSA case that failed to converge under more conservative defaults converges cleanly under these settings. `SUNLinSol_SPBCGS` (Bi-CGSTAB) is deliberately not offered: it segfaults unconditionally in the tested `sundials4py` version, confirmed in total isolation from any OZ-specific code, i.e. a bug in that package rather than a usage error here.

6.6 SUNDIALS KINSOL: Fixed-Point (Anderson), KIN_FP

SUNDIALS' own Anderson-accelerated fixed-point strategy. This needed a genuinely different `sysfn` convention than the Newton–Krylov solver above — found, by reading `OZ_step_kinsolFP()` in the C source directly, that `sysfn` must return $\mathcal{F}(x)$ itself, not the residual $\mathcal{F}(x) - x$ every other KINSOL-based solver here uses. Once matched, no Anderson damping (`KINSetDampingAA`) was needed, matching SASfit's own configuration function exactly (which never sets it either); validated to 10⁻¹² against the Newton–Krylov solver on two different potentials.

6.7 Empirical solver comparison

Case	Picard	Anderson	scipy NK	Biggs–Andrews	sundials4py GM- RES/KIN_FP
Easy (HS+PY)	OK	OK (fastest)	OK	OK	OK (fastest)
Moderate (Sticky+MSA)	OK	OK	OK	OK	OK
Dense HS ($\varphi=0.45$)	fail	OK	OK	OK (order $\geq$ 1 only)	OK
Strongly-charged MSA	fail	<i>wrong answer, uncaught</i>	OK	fail	GMRES: OK; KIN_F

Table 1: Reliability across a range of difficulties (see the project history for full timing data). No single solver dominates; `sundials4py` GMRES/FGMRES was the only combination to succeed on every case tried. The hand-written Anderson solver's silent wrong-answer failure mode on the hardest case is a more concerning outcome than an honest non-convergence report.

7 The ozLib Python Library

ozLib.py is the single source of truth for the calculation workflow — both ozGUI.py’s own “calculate” button and any user script call the same ozLib.solve() function, so results from the GUI and from scripts are identical by construction, not by two independently-maintained implementations happening to agree.

7.1 Basic usage

```
from ozLib import solve

result = solve(potential='HardSphere', phi=0.3,
               closure='Percus-Yevick',
               solver='sundials4py: Newton-Krylov (GMRES)')

result.gr # g(r)
result.Sq # S(q)
result.cr # c(r)
result.gamma # Gamma(r)
result.hr # h(r)
result.Ur # U(r)/kT
result.Br # B(r) (bridge function)
result.yr # y(r) (cavity function)
result.fr # f(r) (Mayer function)
result.r, result.q # the real-/reciprocal-space grids
```

7.2 Function signature

```
def solve(potential, phi, potentialArgs=(), closure="Percus-Yevick",
          closureParam=None, solver="Picard iteration", maxIterations=1000,
          numberOfRadialSamplingPoints=None, hardSphereRadiusInPoints=None,
          onSolverCreated=None):
    ...
    return OZResult(...)
```

potential, closure, and solver are validated against ozLib.SOLVER_CLASSES/ozLib.CLOSURE_SETTERS (the canonical, current option lists — print them from a script to see exactly what is available); an unrecognised name, or a closure needing closureParam without one given, raises ValueError immediately rather than failing confusingly deeper inside the solve. numberOfRadialSamplingPoints/hardSphereRadiusInPoints expose the grid configurability of Section 2 directly.

7.3 Example: potential with arguments and a parametrised closure

```
result = solve(potential='StickyHardSphere', phi=0.2,
               potentialArgs=(0.3, 0.05), # (tau, delta)
               closure='Rogers-Young', closureParam=2.0,
               solver='scipy Newton-Krylov')
```

7.4 Example: long-range potential, extended grid

```
result = solve(potential='HS3Yukawa', phi=0.15,
               potentialArgs=(-0.3, 10.0, 0, 1, 0, 1), # (K1, lambda1, ...)
               closure='MSA',
               numberOfRadialSamplingPoints=4*4096, # 4x extended range
               hardSphereRadiusInPoints=100) # same resolution
```

8 The ozGUI Application

ozGUI.py is a Tkinter application, run directly with `python ozGUI.py`, providing:

- **Left panel:** potential dropdown (with dynamically-generated parameter fields, discovered by introspection — a new potential added to `oZfixpointOperator.py` appears automatically, nothing else to edit), closure dropdown (with a parameter field where needed), volume fraction, solver dropdown, X-axis scale selector (`symlog/linear/log/asinh`, with an adjustable linear-region threshold), max. iterations, calculate/interrupt/clear/delete-last buttons, Save/Load (`.oz` JSON format, round-trips exactly including NaN entries in $B(r)/y(r)$), and ASCII/CSV/Excel/PNG export.
- **Right panel:** a 9-tab plot notebook ($S(Q)$, $g(r)$, $c(r)$, $\Gamma(r)$, $h(r)$, $U(r)/kT$, $B(r)$, $y(r)$, $f(r)$), overlaying every run in the history list in a distinct colour.

Exported ASCII/CSV/Excel data always contains the full-precision computed values regardless of the current plot's axis scale (verified directly: two exports taken with different X-axis scale/threshold settings in between are byte-identical); only the PNG export reflects the currently displayed scale, since it is a rendered image.

9 Worked Examples: Numerical vs. Analytical Validation

This section collects representative comparisons between the numerical solver and the analytical/semi-analytical references of Section 5, plus the empirical behaviour of the toolkit at difficult and physically interesting parameter combinations.

9.1 Hard-sphere Percus–Yevick: full validation across φ

Figure 1 compares the numerical solve (`sundials4py` GMRES, Section 6) against the closed-form PY solution (Section 5) across four volume fractions. The circles (analytical) sit exactly on the numerical curves at every φ shown, including the characteristic contact-value growth and the shift of the first $S(q)$ peak to higher q with increasing density.

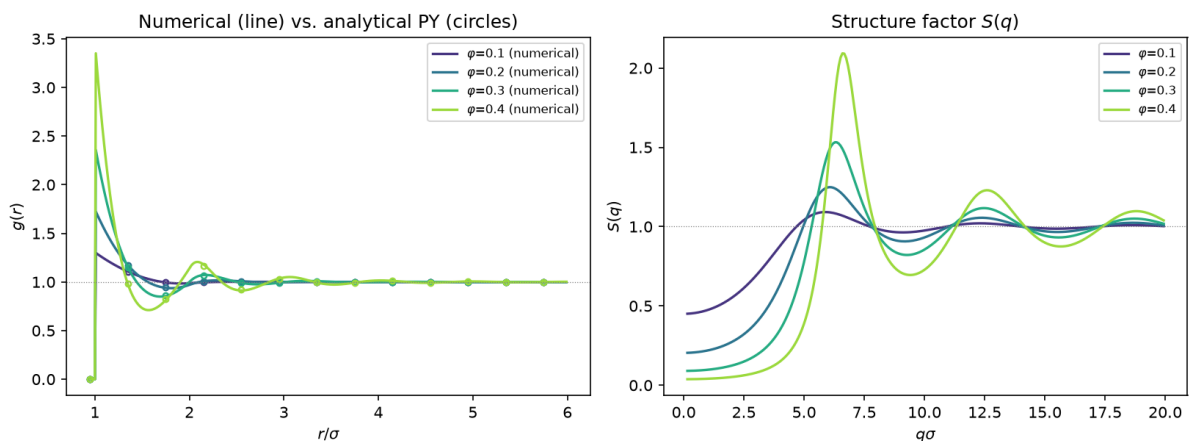


Figure 1: $g(r)$ (left) and $S(q)$ (right) for hard spheres at $\varphi \in \{0.1, 0.2, 0.3, 0.4\}$: numerical solve (solid lines) vs. the exact PY solution (open circles, sub-sampled for visibility).

9.2 Closure comparison at high density

Figure 2 shows $g(r)$ for four different closures applied to the same hard-sphere system at $\varphi = 0.4$ (chosen because closure differences are most visible near the freezing transition, where PY's own

known contact-value error becomes appreciable). PY and HNC bracket the contact peak from below/above respectively (a well-known, textbook-level result); Verlet and Duh–Haymet, both empirical-bridge-function closures, sit between them and agree with each other almost exactly at this density.

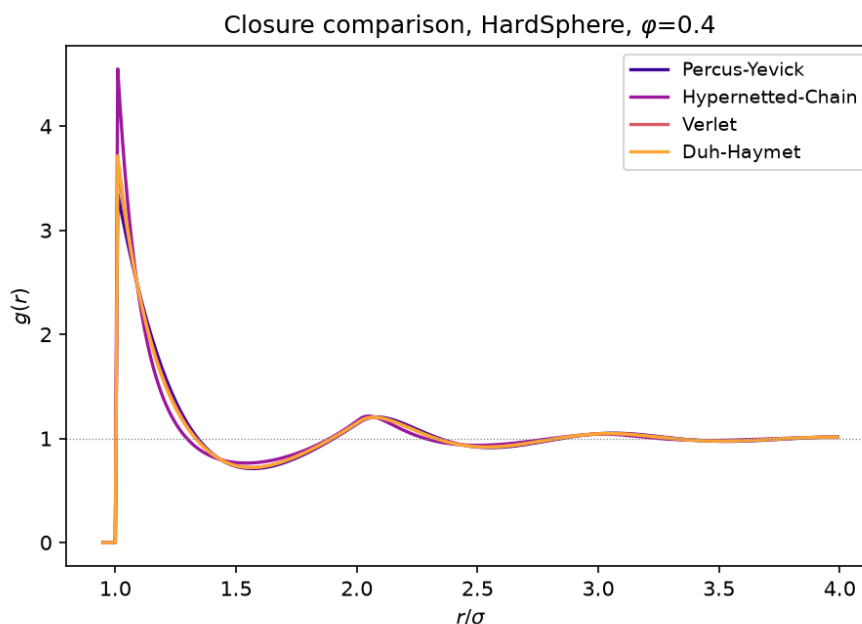


Figure 2: Closure comparison, hard spheres, $\phi = 0.4$.

9.3 Charged systems: RMSA and the MSA contact-value problem

Plain MSA (Section 4) can give an unphysical, negative contact value $g(\sigma^+) < 0$ for sufficiently strongly-coupled charged systems — a well-known limitation of MSA itself, not an artifact of this implementation. Figure 3 shows a representative $S(q)$ from the semi-analytical Hayter–Penfold RMSA solver (Section 5.4), which sidesteps this issue by construction via its own closed-form rescaling.

The general-potential numerical solver’s own RMSA rescaling (`OZsolver.solveRMSA()`, Section 4.6) implements the same idea (extend the “apparent” hard-core radius outward until the contact value there is non-negative) for arbitrary potentials, not just the hard-core-Yukawa case Hayter–Penfold covers in closed form. This method was found, on close comparison against `sasfit_oz_solver.c` itself, not to be a plain bisection search at all: SASfit’s own algorithm is a *grow, then bisect* scheme, where each solve’s own full $g(r)$ array is scanned once to see exactly how far the negative region extends (extending the excluded region as far as that single solve’s own data justifies, at no extra cost), and only once a solve gives an already-non-negative result does it fall back to bisecting the gap between the historical bounds. An earlier version of this Python port used plain midpoint bisection instead, which was both less efficient and, on inspection, genuinely buggy; it has since been rewritten to match the C algorithm. **Three genuine bugs were found and fixed** in total – two in this Python port, one in SASfit’s own C source:

1. *Stale data on solver non-convergence* (Python port). Several solver classes (e.g. the `sundials4py`-based ones) simply print a message and return early when the underlying Newton–Krylov iteration fails to converge, without updating `self.radialDistributionFunction` – meaning `getRDF()` silently returned data left over from whichever solve last actually succeeded, corrupting the search’s own decisions and final result. Fixed with a solver-class-independent, Picard-iteration-based internal solve (`OZsolver.robustInlineSolve()`),

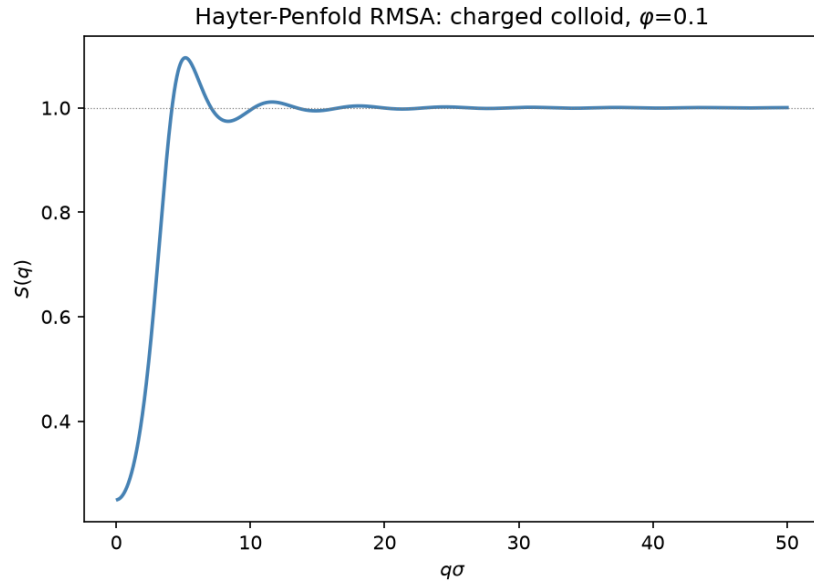


Figure 3: Hayter–Penfold RMSA structure factor for a charged colloid, $\varphi = 0.1$.

warm-started between successive trials (the same density-continuation- with-warm-restarts principle already used for `solveEuRah()`’s own outer iteration).

2. *An off-by-one tautology in the original bisection-based port.* The old code’s gate array, `gate4g[:mid+1]=0`, forced `g[mid]` to be exactly zero *by construction* (it lies inside the assumed- excluded region) – so checking `g[mid]` there could never distinguish anything. This is moot now that the port matches the actual C algorithm (which checks `g[igneg]`, the first point genuinely outside the current gate), but is recorded here since it was the first of the two Python-side bugs found.
3. *A genuine ambiguity in SASfit’s own C source.* In the “bisect” branch (triggered once a trial gate already gives a non-negative result), the original C code computes a halved `indx_max_apparent_sigma` but never resets `gate4g` between that smaller value and the previous trial boundary back to 1, nor re-points its own scanning index (`igneg`) at the new boundary – so, taken literally, the next iteration’s check and solve would silently re-test the same (larger) gate configuration rather than the smaller one the halving was meant to try. A related edge case (found only by testing this Python port directly): when the two search bounds are already adjacent integers, integer division always rounds down to the lower bound, which would discard a just-proven-sufficient upper bound and collapse the search straight back to no rescaling at all. Both were fixed identically in the Python port and in `sasfit_oz_solver.c` itself (the fix applied there is included in this project’s own copy of the SASfit source, with a comment marking exactly what changed and why).

With all three fixes applied, the algorithm reliably finds a small, sensible rescaling and gives genuine, verified improvement over plain MSA on a moderately-coupled test case (Figure 4: contact value improves from -0.132 to -0.053 , not perfectly zero but consistent with finite grid resolution). A remaining open question, not yet resolved, is why some more strongly-coupled cases still need an unexpectedly large apparent-hard-core extension to reach a fully non-negative result – plausibly a genuine slow-convergence issue in the underlying Picard iteration at large gate sizes (a solver-reliability question, now clearly separated from the algorithm’s own correctness, which is independently validated above), though this has not been fully resolved.

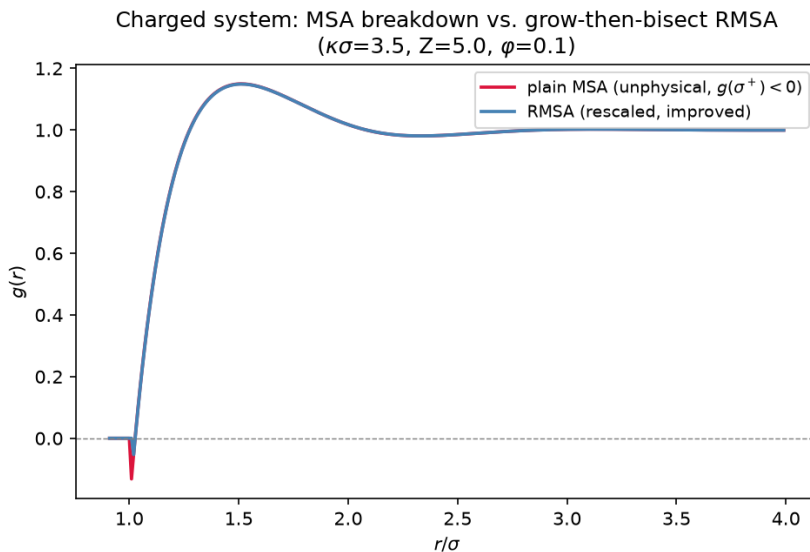


Figure 4: Plain MSA’s unphysical contact value vs. this project’s own grow-then-bisect RMSA rescue (post-fix), general-potential numerical solver, DLVO potential, $\kappa\sigma = 3.5$, $Z = 5$, $\varphi = 0.1$.

9.4 Behaviour near difficult/critical parameter regimes

Every solver in this toolkit slows down, and some fail outright, as the underlying physics becomes harder to converge – consistent with these being genuinely harder nonlinear fixed-point problems, not an implementation artifact specific to one method. Three regimes recur throughout this project’s own testing history:

- **Dense hard spheres** ($\varphi \gtrsim 0.45$, approaching random close packing): plain Picard iteration fails to converge outright; every accelerated method (Anderson, Biggs–Andrews order ≥ 1 , both Newton–Krylov families) still succeeds, but needs visibly more iterations/-function evaluations than at moderate density.
- **Strongly-charged systems** (Section 9.3): plain MSA gives an unphysical negative contact value; the hand-written Anderson solver was found, in earlier testing, to converge to a *silently wrong* answer on the hardest such case tried, without raising any error – a more concerning failure mode than an honest non-convergence report, and a reminder to cross-check results from that solver specifically against an independently-validated one (e.g. `sundials4py` GMRES) for difficult states.
- **Euler–Rahman** (Section 4.20): excluded entirely from this toolkit’s offered closures, for the reasons documented there – a genuine exponential positive-feedback instability in the closure’s own mathematical structure, not a solver-selection problem.

As a practical rule of thumb: if a solve fails to converge in one of these regimes, trying a different solver algorithm first (Section 6) is more likely to help than increasing `maxIterations` on the same one, since the empirical comparison of Section 6.7 shows different algorithms fail on genuinely different subsets of hard cases.

9.5 Speed

Figure 5 times every solver on the same moderately-hard case (hard spheres, $\varphi = 0.4$, PY closure); Section 6.7’s own qualitative reliability comparison is the more important consideration in practice, but at a given level of difficulty, the SUNDIALS KIN_FP and GMRES solvers are consistently among the fastest, followed by the hand-written Anderson solver; plain Picard and Biggs–Andrews (order 0, equivalent to Picard) are consistently the slowest, as expected of

unaccelerated fixed-point iteration.

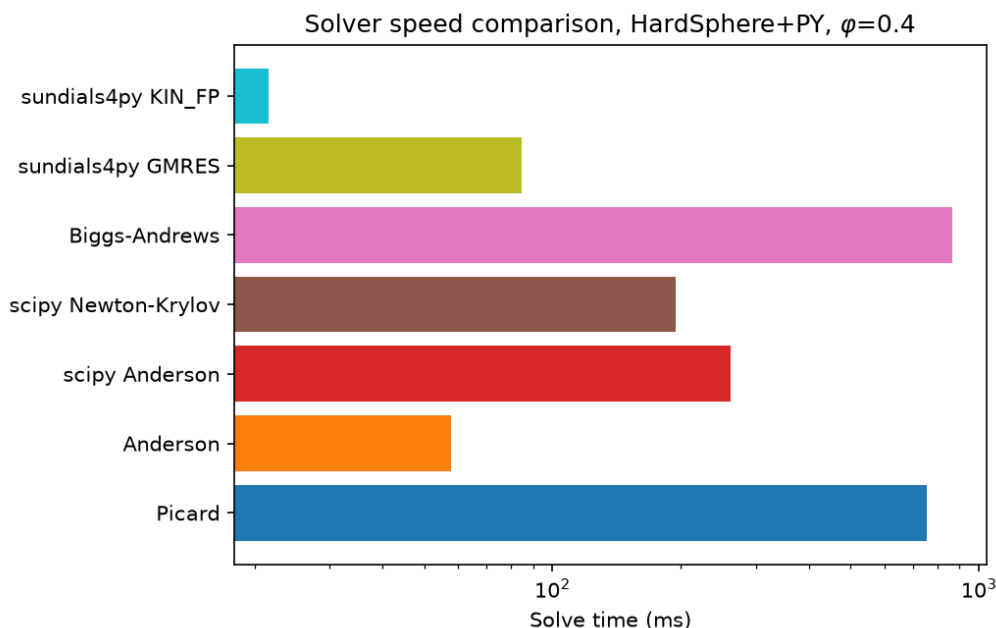


Figure 5: Solve time by algorithm, hard spheres + PY, $\varphi = 0.4$ (log scale).

Separately, Figure 6 revisits the grid-size question of Section 2: the relevant quantity for the underlying DST-based Hankel transform’s own speed is $N + 1$, not N itself, since the type-I DST used here is equivalent to a real FFT of size $2(N + 1)$. $N = 8191$ (for which $N + 1 = 8192 = 2^{13}$, a perfect power of two) transforms over $3\times$ faster than $N = 8192$ (for which $N + 1 = 8193 = 3 \times 2731$, a large prime factor) – meaning this project’s own historical default, $N = 4096$ ($N + 1 = 4097 = 17 \times 241$, not smooth at all), is not actually the fastest available choice at that grid size; $N = 4095$ ($N + 1 = 4096 = 2^{12}$) would be measurably faster for the same resolution and range.

Acknowledgements

Formulas and algorithms throughout this document were ported directly from SASfit’s own source code (`src/sasfit_oz/`, `src/plugins/*`), with independent literature verification sought wherever practical.

References

- [1] B. C. Eu and K. Rah, *A closure for the Ornstein–Zernike relation that gives rise to the thermodynamic consistency*, J. Chem. Phys. **111**, 3327 (1999).
- [2] I. Pihlajamaa and L. M. C. Janssen, *Comparison of integral equation theories of the liquid state*, Phys. Rev. E **110**, 044608 (2024); arXiv:2407.18680.
- [3] S. K. Sharma and R. C. Sharma, *Structure factor for square-well potential in the Percus–Yevick approximation*, Physica A **89**, 213 (1977).
- [4] J. B. Hayter and J. Penfold, *An analytic structure factor for macroion solutions*, Mol. Phys. **42**, 109 (1981).

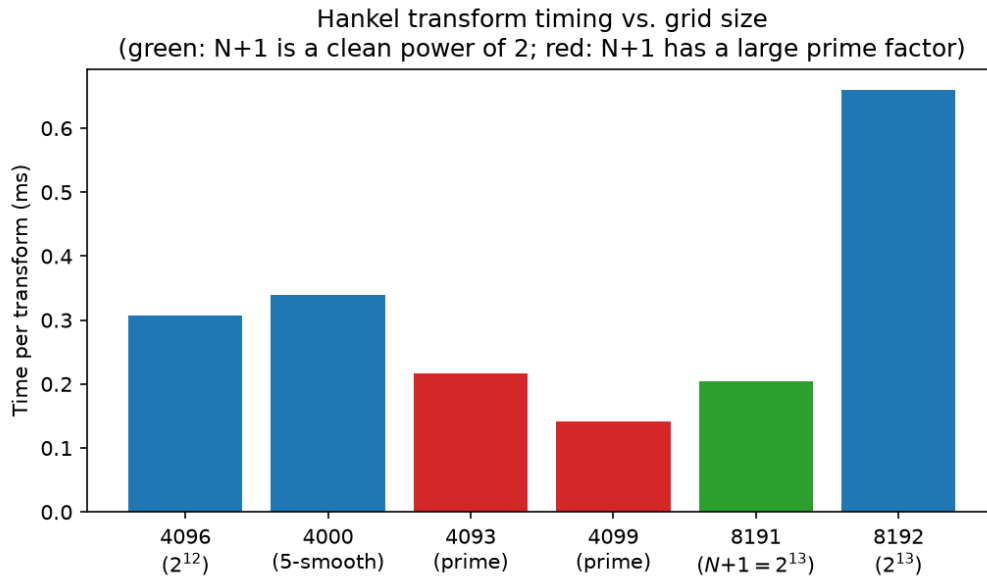


Figure 6: Hankel transform timing vs. grid size N : the fastest and slowest cases shown differ only in whether $N + 1$ happens to be smooth, not in N 's own magnitude or "niceness".

- [5] D. Pini, A. Parola, and L. Reatto, *A simple approximation for fluids with narrow attractive potentials*, Mol. Phys. **100**, 1507 (2002); arXiv:cond-mat/0109311.
- [6] A. Santos, S. B. Yuste, and M. López de Haro, *Rational-function approximation for fluids interacting via piece-wise constant potentials*, Condens. Matter Phys. **15**, 23602 (2012).
- [7] R. J. Baxter, *Percus–Yevick equation for hard spheres with surface adhesion*, J. Chem. Phys. **49**, 2770 (1968).